Jira-Based Manual Testing, Test Execution, and Defect Analysis Report for Event Management System

KASHAF FATIMA

kashaf.fatimaa132@gmail.com

+92 3466208273

Table of Contents

Introduction

This document presents a comprehensive Software Quality Assurance (SQA) testing report conducted for an Event Management System. The primary objective of this testing effort was to evaluate the application's functionality, reliability, usability, and compliance with specified requirements through systematic manual testing practices.

The testing process was managed using Jira, which was utilized for test case management, test execution tracking, defect reporting, and progress monitoring. A total of 47 test cases were designed and executed across multiple modules of the application, including Authentication, Event Management, Meeting Management, Member Management, Profile Management, Event Media Management, and Request Management.

The report includes test execution dashboards, bug tracking dashboards, detailed test case results, defect analysis, and recommendations for identified issues. Each defect has been documented with its impact on the application and proposed solutions to improve system quality and user experience.

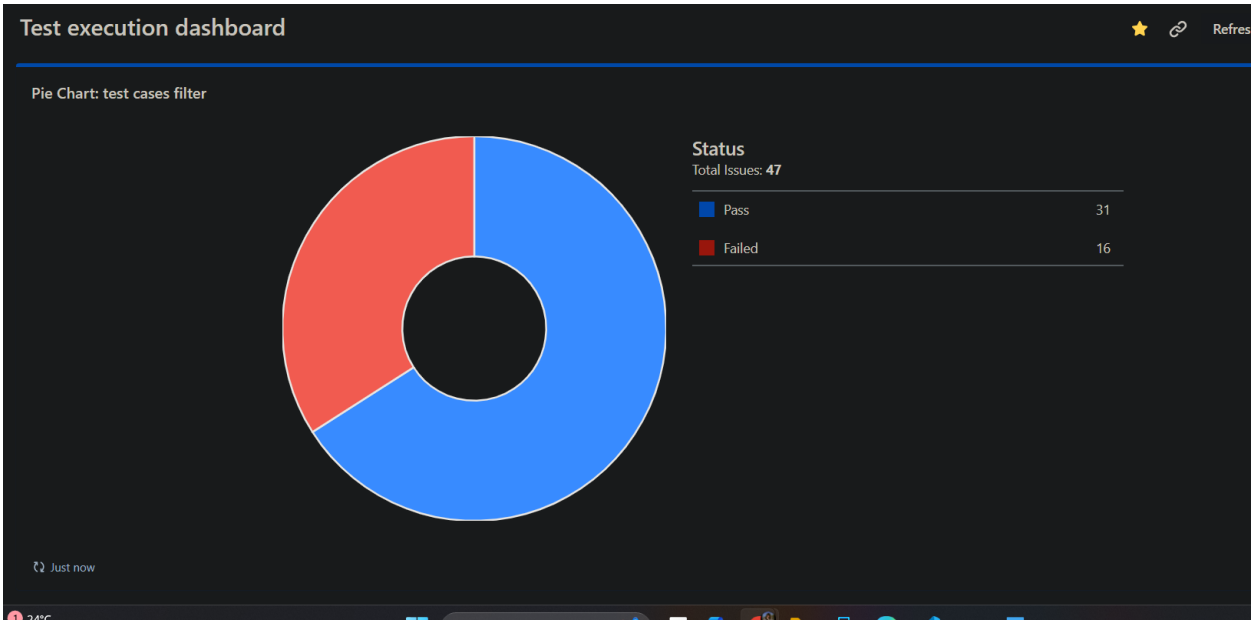
The purpose of this document is to demonstrate a structured QA approach, showcasing skills in test planning, test execution, defect identification, bug reporting, and quality assessment. Additionally, it highlights the practical use of Jira in managing software testing activities and maintaining traceability throughout the testing lifecycle.

This report serves as both a quality assessment of the application and a portfolio project demonstrating industry-standard Software Quality Assurance practices.

1. Jira Dashboard

a. Test execution dashboard

I. Pie chart showing test case status (Passed/Failed/Blocked/etc.)

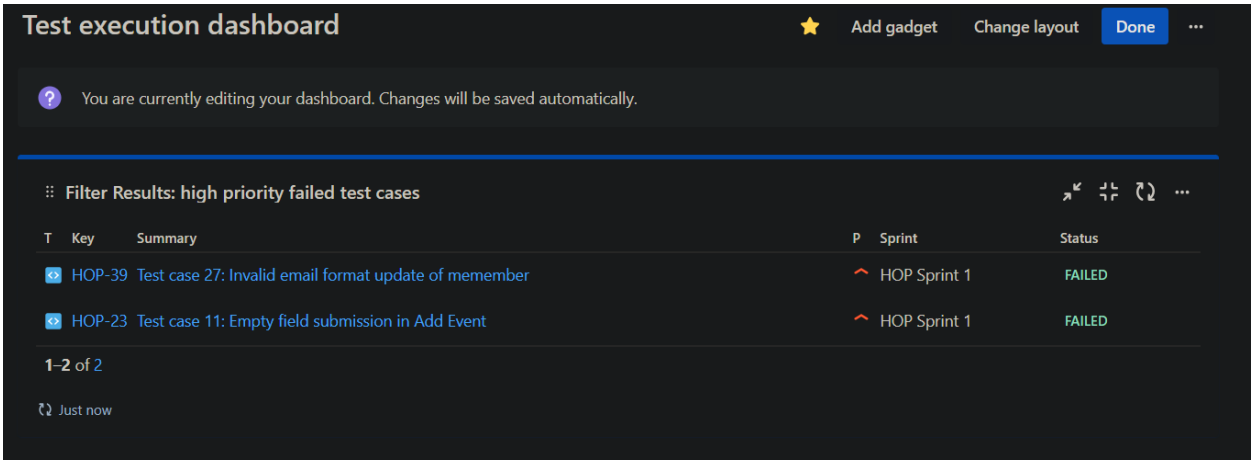


This pie chart shows that 16 test cases out of 47 are failed when they were executed while 31 passes. This shows that our project still needs a lot of improvement.

II. List of recently executed test cases

This shows the list of test cases that were executed recently. We can use this filter to check which test case was recently executed that was passed or failed. If one of test case fails, then it will be most likely helpful in finding the recently failed test case.

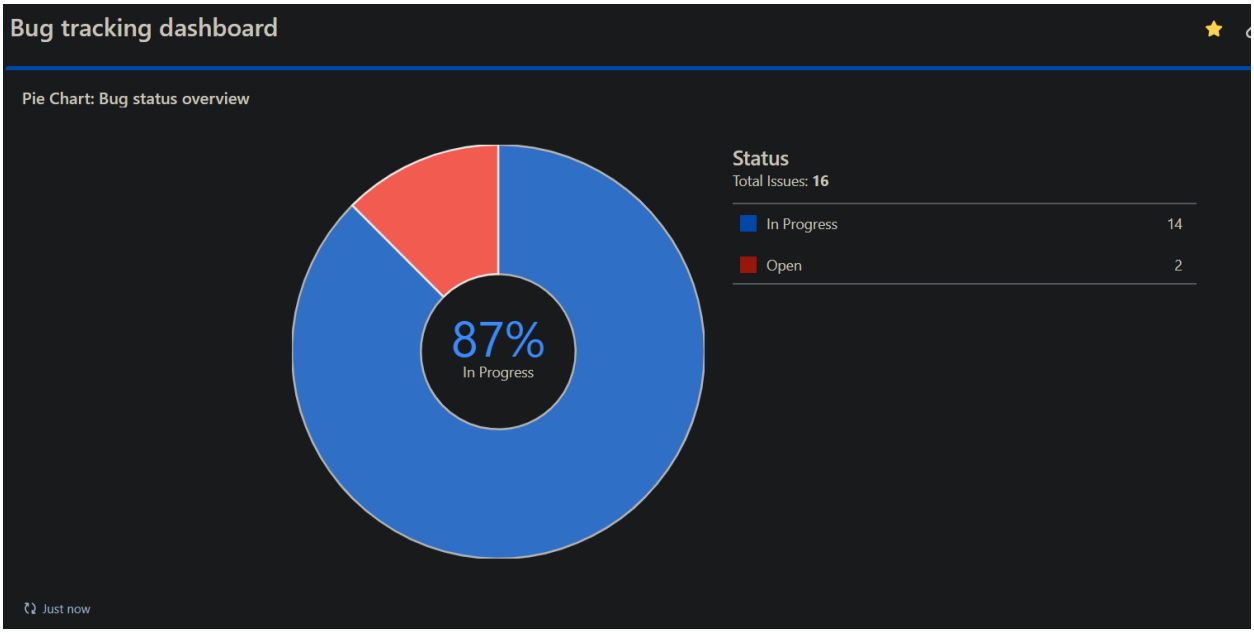
III. List of high-priority failed test cases



This filter will help in finding test cases that re highly prioritized test cases that were executed but were failed.

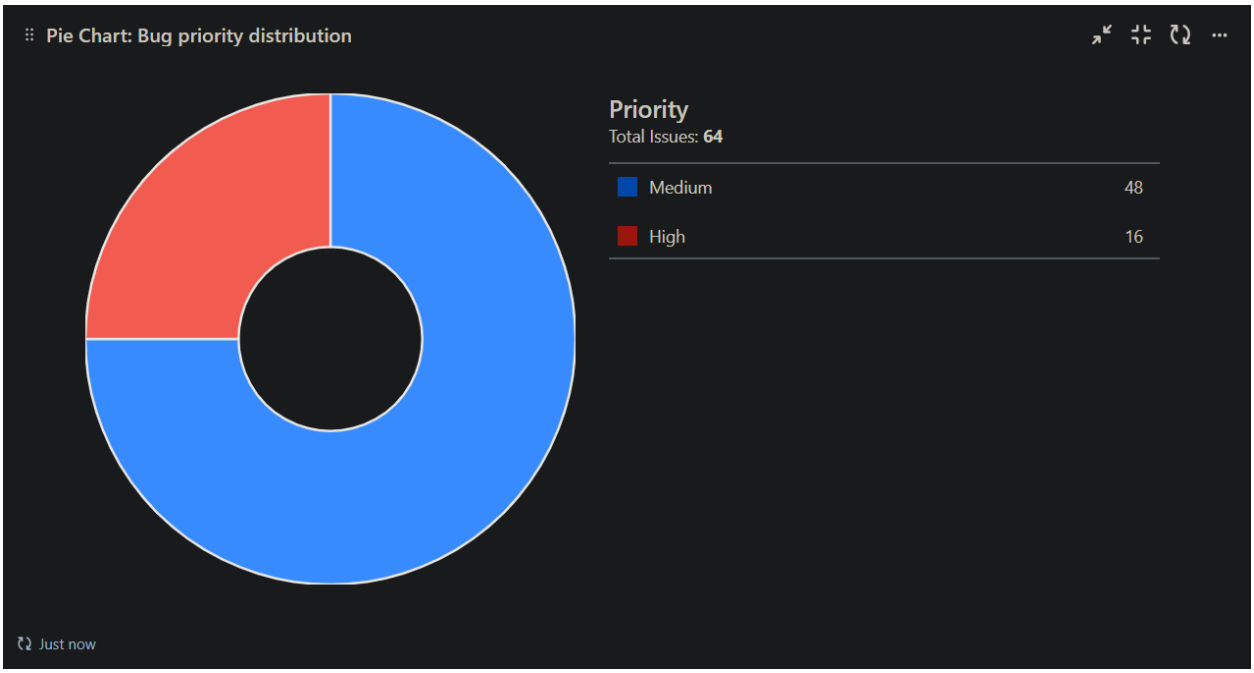
Bug Tracking Dashboard:

I. Bug status overview (Open/In Progress/Resolved/etc.)



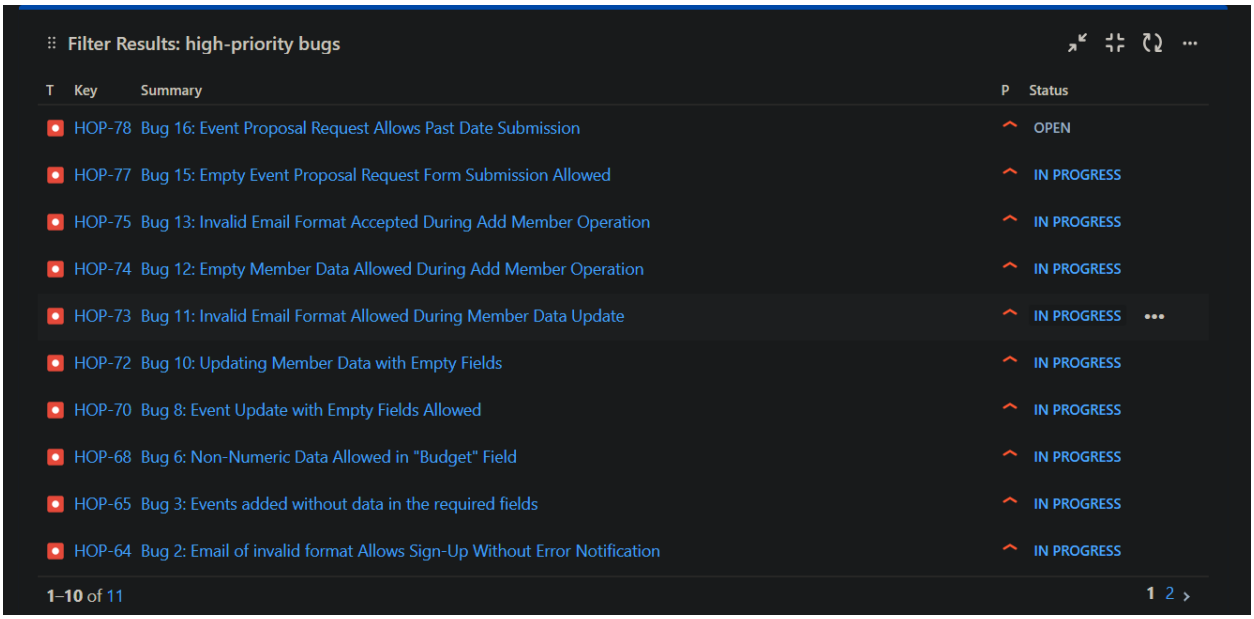
This pie chart will help in overviewing the status of bugs. It will display a ratio of the number of bugs in each category of status.

II. Bug priority distribution



This pie chart will help in finding the total number of bugs in each priority category. It will show the ratio of categories

III. List of critical and high-priority bugs



This filter will help in finding bugs that are critical or high-priority. This can be helped in a way when a developer has to firstly resolve all critical and high priority bugs, he can then use this filter to find those bugs.

2. Analysis of test execution results, including pass/fail ratios and identified trends

Test case 1: Empty submission of email and password in login

Summary:

The test case will check behavior of the app when user tries to login without entering any data in email and password text fields.

Description:

1. Open Nimbus app
2. Press “Log in“ button to navigate to login page
3. Enter nothing in email and password text fields.
4. Press “Login“ to navigate to homepage.

Expected Result:

The app should not allow user to login and displays an error message for asking to enter email and password.

Actual result:

When user press login, Nimbus app does not allow user to navigate to home page by showing an error message "*Invalid email format*".

Status: Pass

Test case 2: Login using invalid email format

Summary:

Test case checks the behavior of app when user tries to login using an email that has wrong email format.

Description:

1. Open Nimbus app
2. Press "Log in" button to navigate to login page
3. Enter email and password
4. Press "Login" button

Test Data:

1st input:

- Email: kas@gmail
- Password: 12345678Aa$

2nd input:

- Email: kasgmail@.com
- Password: 12345678Aa$

3rd input:

- Email: kasgmail@com
- Password: 12345678Aa$

Expected Result:

App should not let user to login using wrong format email and display an error message like "*Invalid email format*".

Actual result:

When user enter wrong format email and press login, Nimbus app does not allow user to navigate to home page by showing an error message "*Invalid email format*".

Status: Pass

Test case 3: Login using incorrect password

Summary:

Test case checks the behavior of app when user tries to login using incorrect password.

Description:

1. Open Nimbus app
2. Press "Log in" button to navigate to login page
3. Enter email and password
4. Press "Login" button

Test Data:

1st input:

- Email: kas@gmail.com
- Password: 12345678

2nd input:

- Email: kas@gmail.com
- Password: 98765

3rd input:

- Email: kas@gmail.com
- Password: 87654321Aa$

Expected Result:

App should not let user to login using wrong password and display an error message like "*Invalid email or password*".

Actual result:

When user enter incorrect password and press login, Nimbus app does not allow user to navigate to home page by showing an error message "*Invalid email or password*".

Status: Pass

Test case 4: Login using google authentication

Summary:

This test case checks the behavior of app when user tries to login using google authentication.

Description:

1. Open Nimbus app
2. Press “Log in“ button to navigate to login page.
3. Press “Google icon” for login using google authentication.
4. Select an email from the popup menu.
5. Homepage will be opened.

Test Data:

Google account: kashaf1029@gmail.com

Expected Result:

When user press google login, a pop up menu for selecting email should appear. Upon selection of the email user should be directed to homepage of the role of which email is.

Actual result:

When user press google login, a pop up menu for selecting email appeared. Upon selecting of the email user is navigated to guest home page.

Status: Pass

Test case 5: Reset password of email on which account does not exists

Summary:

This test case checks the behavior of the app if user wants to reset the password of an email on which account does not exists.

Description:

1. Open Nimbus app

2. Press “Log in“ button to navigate to login page
3. Press “forget password” button
4. Enter email in text field and press “Send reset link“

(A reset link will be sent in Email)

Test Data:

Email: sidra@gmail.com

Expected Result:

When user tries to reset password of an email on which account does not exists in database, app should not send an reset password email and show message “Account does not exist on this email!“.

Actual result:

When user tries to reset password of an email on which account does not exists, app sends a reset password email and show message “Password reset email sent!“ when user press “Send reset link“ button.

Status: Fail

Test case 6: Reset password of email on which account exists

Summary:

This test case checks the behavior of the app if user wants to reset the password of an email on which account exists.

Description:

1. Open Nimbus app
2. Press “Log in“ button to navigate to login page
3. Press “forget password” button
4. Enter email in text field and press “Send reset link“

(A reset link will be sent in Email)

Test Data:

Email: kashaf1029@gmail.com

Expected Result:

When user tries to reset password of an email on which account exists in database, app should send a reset password email and show message “Password reset email sent!!“. User should receive a link in email.

Actual result:

When user tries to reset password of an email on which account exists, app sends a reset password email and show message “Password reset email sent!“ when user press “Send reset link“ button.

Status: Pass

Test case 7: Empty field submission in sign up

Summary:

The test case will check behavior of the app when user tries to sign up without entering any data in Email, Username and Password text fields.

Description:

1. Open Nimbus app
2. Press “Sign up“ button to navigate to login page
3. Enter nothing in email, username and password text fields.
4. Press “Signup“ to navigate to homepage.

Expected Result:

The app should not allow user to sign up and displays an error message for asking to enter email, username and password.

Actual result:

When user press “Sign up”, Nimbus app does not allow user to navigate to home page by showing an error message “*Enter valid name. Your name should not contain any numbers or special characters*“.

Status: Pass

Test case 8: Weak password format for creating account

Summary:

This test case checks the behavior of Nimbus app when user creates a new account but sets a weak password. The threshold for password is :

- It should be greater than 8 characters
- It must contain at least 1 capital letter
- It must contain at least 1 small letter
- It must contain at least 1 special character
- It must contain at least 8 number

Description:

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Enter valid username , email and password.
4. Select the role from drop down menu.
5. Press “Sign up“ button.

Test Data:

- Username: Huda
- Email: "huda@gmail.com"
- Password: "12a45"
- Role: "Members"

Expected Result:

When user tries to sign up by using a weak password that does not follow the criteria, the app should not allow it by displaying an error message “Your password must contain: at least 1 uppercase letter, 1 lowercase letter, 1 special character, and 8 numeric digits“

Actual result:

When user tries to signup using a weak password, app does not allows and show error message “Your password must contain: at least 1 uppercase letter, 1 lowercase letter, 1 special character, and 8 numeric digits“

Status: Pass

Test case 9: Signup by entering numbers in username

Summary:

This test case checks the behavior of Nimbus app when user creates a new account using numbers as username. The threshold for password is :

- It should not contain any numbers
- It should not contain any special characters

Description:

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Enter valid username , email and password.
4. Select the role from drop down menu.
5. Press “Sign up“ button.

Test Data:

- Username: 1234
- Email: "huda@gmail.com"
- Password: "12345678Aa$"
- Role: "Members"

Expected Result:

When user tries to sign up by using numerical number as username that does not follow the criteria, the app should not allow it by displaying an error message “Enter valid username. Your name should not contain any numbers or special characters“

Actual result:

When user tries to signup using numerical number as username, app does not allows and show error message “Enter valid username. Your name should not contain any numbers or special characters“

Status: Pass

Test case 10: Invalid email input for sign up

Summary:

This test case checks the behavior of Nimbus app when user creates a new account but enters email of invalid format.

Description:

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Enter valid username , email and password.
4. Select the role from drop down menu.
5. Press “Sign up“ button.

Test Data:

- Username: Huda
- Email: "1234@gmail.com"
- Password: "12345678Aa$"
- Role: "Members"

Expected Result:

When user tries to sign up by using email that does not follow the criteria, the app should not allow it by displaying an error message “Invalid email. Your email should not start with a number“

Actual result:

When user tries to signup using an email that does not follow requirements, app allows it and user is added in database. App also does not notify any issue to user by displaying any error message.

Status: Fail

Test case 11: Empty field submission in Add Event

Summary:

The test case will check behavior of the app when user tries to add an event without entering any data in text fields.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.

4. Leave all text fields empty and try to submit the form.
5. Press “Add“ button.

Test Data:

No input (leave all fields empty).

Expected Result:

The app should not allow user to Add an empty event and displays an error message for asking to enter data in all the required fields.

Actual result:

When user tries to add an event that has nothing in its text fields, the app allows him to do so. When user press “Add” button event is added to database and a popup message informs user “Event is added!“.

Status: Fail

Test case 12: Add an event in past

Summary:

This test case will check the behavior of app when user tries to add an event in past by a previous date.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Select a date from past and tries to submit it.
5. Press “Add“ button.

Test Data:

Date: 2024-09-02

Expected Result:

When user tries to add an event in past by choosing a previous date from present, then app should not allow user and display some error message that says “Event can be added in past!“.

Actual result:

When user tries to add an event in past by choosing a previous date from present, app allows him to do so. Event is then added in database.

Status: Failed

Test case 13: Exceeding limit of 500 words for event Description

Summary:

This test case check the behavior of app if user add 500 words in text field option “Event Description“ in “Add event” module.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Enter 502 words in “Event Description“ text field.
5. Enter valid data in all text fields that is required.
6. Press “Add“ button.

Test Data:

Event Name: Orientation

Date: 2024-09-16

Time: 12:00 PM

Organizer: FSES

Budget: 50,000

Venue: Auditrium

Event Description: “Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem

ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, quis gravida magna mi a libero. Fusce vulputate eleifend sapien. Vestibulum purus quam, scelerisque ut, mollis sed, nonummy id, metus. Nullam accumsan lorem in dui. Cras ultricies mi eu turpis hendrerit fringilla. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; In ac dui quis mi consectetuer lacinia. Nam pretium turpis et arcu. Duis arcu tortor, suscipit eget, imperdiet nec, imperdiet iaculis, ipsum. Sed aliquam ultrices mauris. Integer ante arcu, accumsan a, consectetuer eget, posuere ut, mauris. Praesent adipiscing. Phasellus ullamcorper ipsum rutrum nunc. Nunc nonummy metus. Vestibulum volutpat pretium libero. Cras id dui. Aenean ut eros et nisl sagittis vestibulum. Nullam nulla eros, ultricies sit amet, nonummy id, imperdiet feugiat, pede. Sed lectus. Donec mollis hendrerit risus. Phasellus nec sem in justo pellentesque facilisis. Etiam imperdiet imperdiet orci. Nunc nec neque. Phasellus leo dolor, tempus non, auctor et, hendrerit quis, nisi. Curabitur ligula sapien, tincidunt non, euismod vitae, posuere imperdiet, leo. Maecenas malesuada. Praesent congue erat at massa. Sed cursus turpis vitae tortor. Donec posuere vulputate arcu. Phasellus accumsan cursus velit. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Sed aliquam, nisi quis porttitor congue, elit erat euismod orci, ac placerat dolor lectus quis orci. Phasellus consectetuer vestibulum elit. Aenean tellus metus, bibendum sed, posuere ac, mattis non, nunc. Vestibulum fringilla pede sit amet augue. In turpis. Pellentesque posuere. Praesent turpis. Aenean posuere, tortor sed cursus feugiat, nunc augue blandit nunc, eu sollicitudin urna dolor sagittis lacus. Donec elit libero, sodales nec, volutpat a, suscipit non, turpis. Nullam sagittis. Suspendisse pulvinar, augue ac venenatis condimentum, sem libero volutpat nibh, nec pellentesque velit pede quis nunc. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Fusce id purus. Ut varius tincidunt libero. Phasellus dolor. Maecenas vestibulum mollis diam. Pellentesque“ (502 words)

Expected Result:

When user tries to enter more than 500 words in “Event description“ in Add event module, the app should not allow user to cross the limit and show an error message.

Actual result:

When user tries to enter more than 500 words in “Event description“ in Add event module, user can only enter 73 words as shown in the picture. Text field does not allow to enter further words

Status: Fail

Test case 14: Submission of budget with non-numeric input

Summary:

This test case check the behavior of app if user add non-numeric data in text field option “Budget“ in “Add event” module.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Add non-numeric data in “Budget“ text field.
5. Enter valid data in all text fields that is required.
6. Press “Add“ button.

Test Data:

Event Name: Orientation

Date: 2024-09-16

Time: 12:00 PM

Organizer: FSES

Budget: fiftey thousand

Venue: Auditrium

Event Description: “Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, quis gravida magna mi a libero. Fusce vulputate eleifend sapien. Vestibulum purus quam, scelerisque ut, mollis sed, nonummy

id, metus. Nullam accumsan lorem in dui. Cras ultricies mi eu turpis hendrerit fringilla. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; In ac dui quis mi consectetuer lacinia. Nam pretium turpis et arcu. Duis arcu tortor, suscipit eget, imperdiet nec, imperdiet iaculis, ipsum. Sed aliquam ultrices mauris. Integer ante arcu, accumsan a, consectetuer eget, posuere ut, mauris. Praesent adipiscing. Phasellus ullamcorper ipsum rutrum nunc. Nunc nonummy metus. Vestibulum volutpat pretium libero. Cras id dui. Aenean ut eros et nisl sagittis vestibulum. Nullam nulla eros, ultricies sit amet, nonummy id, imperdiet feugiat, pede. Sed lectus. Donec mollis hendrerit risus. Phasellus nec sem in justo pellentesque facilisis. Etiam imperdiet imperdiet orci. Nunc nec neque. Phasellus leo dolor, tempus non, auctor et, hendrerit quis, nisi. Curabitur ligula sapien, tincidunt non, euismod vitae, posuere imperdiet, leo. Maecenas malesuada. Praesent congue erat at massa. Sed cursus turpis vitae tortor. Donec posuere vulputate arcu. Phasellus accumsan cursus velit. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Sed aliquam, nisi quis porttitor congue, elit erat euismod orci, ac placerat dolor lectus quis orci. Phasellus consectetuer vestibulum elit. Aenean tellus metus, bibendum sed, posuere ac, mattis non, nunc. Vestibulum fringilla pede sit amet augue. In turpis. Pellentesque posuere. Praesent turpis. Aenean posuere, tortor sed cursus feugiat, nunc augue blandit nunc, eu sollicitudin urna dolor sagittis lacus. Donec elit libero, sodales nec, volutpat a, suscipit non, turpis. Nullam sagittis. Suspendisse pulvinar, augue ac venenatis condimentum, sem libero volutpat nibh, nec pellentesque velit pede quis nunc. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Fusce id purus. Ut varius tincidunt libero. Phasellus dolor. Maecenas vestibulum mollis diam. Pellentesque“ (502 words)

Expected Result:

When user tries to enter non-numeric data in “Budget“ in Add event module, the app should not allow user to add non-numeric data in Budget and show an error message “You can only add numeric data!“.

Actual result:

When user tries to enter non-numeric data in “Budget“ in Add event module, App allows user to do so and event is then saved in data base as shown in pictures.

Status: Fail

Test case 15: Input of numeric data instead of event name

Summary:

This test case check the behavior of app if user add numeric data in text field option “Event name“ in “Add event” module.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Add numeric data in “Event name“ text field.
5. Enter valid data in all text fields that is required.
6. Press “Add“ button.

Test Data:

Event Name: 912

Date: 2024-09-16

Time: 12:00 PM

Organizer: FSES

Budget: 10000

Venue: Auditrium

Event Description: “Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. “

Expected Result:

When user tries to enter numeric data in “Event name“ in Add event module, the app should not allow user to add numeric data in Event Name and show an error message “Event name should not start with number!“.

Actual result:

When user tries to enter numeric data in “Event name“ in Add event module, App allows user to do so and event is then saved in data base as shown in pictures.

Status: Fail

Test case 16: Update event to see changes

Summary:

The test case will check behavior of the app when user tries to update an existing event.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “View event” module.
4. Press “Edit” button on one of the event.
5. Rewrite the data that you want to update.
6. Press “Update“ button.

Test Data: Event Name: 912

Date: 2024-09-16

Time: 12:00 PM

Organizer: FSES

Budget: 10000

Venue: Auditorium

Event Description: Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor elod wfyns wdym hseeh kucf ffo

Expected Result:

The app should allow user to Update an already existing event by updating it on database.

Actual result:

When user tries to update an already existed event, the app allows him to do so. When user press “Update” button event is added to database and a popup message informs user “Successfully updated event!“.

Status: Pass

Test case 17: Delete an event from Event list

Summary:

The test case will check behavior of the app when user tries delete an Event.

Description:

1. Open Nimbus app
2. Sign up in app.

3. Go to “View event” module.
4. Delete a specific event by pressing “Delete“ button.

Test Data:

No input (leave all fields empty).

Expected Result:

The app should allow user to delete event and displays remaining event on screen.

Actual result:

When user tries to delete an event, event is deleted from database, app updates user with the help of popup message “Event deleted successfully!“. Remaining events are then displayed on screen

Status: Pass

Test case 18: Handle empty text field submission

Summary:

The test case will check behavior of the app when user tries to update an event without entering any data in some text fields.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “View event” module.
4. Press “Edit” button on one of the event.
5. Enter data in few fields and leave few empty.
6. Press “Update“ button.

Test Data: Event Name: Orientation

Date: 2024-09-16

Time: 12:00 PM

Organizer:

Budget:

Venue: Auditorium

Event Description:

Expected Result:

The app should not allow user to Update an event having empty text fields and displays an error message for asking to enter data in all the required fields.

Actual result:

When user tries to update an event that has empty text fields, the app allows him to do so. When user press “Update” button event is added to database and a popup message informs user “Successfully updated event!“.

Status: Fail

Test case 19: Handle edge case events

Summary:

This test case will check the behavior of app when it has to display large values of data.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Enter maximum limit of data in each text field.
5. Press “Add“ button.
6. Go to “View Event“ module and check UI.

Test Data:

Event Name: Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor elod wfyns wdym hseeh kucf ffo

Date: 2024-09-16

Time: 12:00 PM

Organizer: FSES

Budget: 999999999999

Venue: Auditrium

Event Description: “Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor”

Expected Result:

When user tries to display exceptionally large amount of data of an event, then there should be no UI overflows.

Actual result:

When user tries to display exceptionally large amount of data of an event, then there is no UI overflows. Data is displayed properly on screen as shown in attached picture.

Status: Pass

Test case 20: Only president can update or edit an event

Summary:

This test case is to check that only president can edit and delete an event in Event log.

Description:

1. Open Nimbus app.
2. Login using president’s id.
3. Go to “View Event“ module and check if edit and delete options present.
4. Open slider “Menu“ by clicking 3 bars at top left corner
5. Sign out from app by clicking “Sign out“
6. Now, login with member’s id
7. Go to “View Event“ module and check if edit and delete options present.

Test Data:

1st entry:

- Email: kas@gmail.com
- Password: 12345678Aa$

2nd entry:

- Email: eman@gmail.com
- Password: 12345678Aa$

Expected Result:

When a president login then he should only see “Edit” and “Delete“ options in ”View Event” module. If anyone other then president logs in then he should not see “Edit“ and “Delete“ options in ”View Event” module.

Actual result:

When a president login then he can see “Edit” and “Delete“ options in ”View Event” module. While when a member logs in, he does not see “Edit“ and “Delete“ options in ”View Event” module.

Status: Pass

Test case 21: Empty meeting data field submission

Summary:

The test case will check behavior of the app when user tries to add a meeting without entering any data in text fields also without selecting date and time.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Meeting schedule board” module.
4. Leave all text fields empty and try to submit the form.
5. Press “Done“ button.

Test Data:

No input (leave all fields empty).

Expected Result:

The app should not allow user to Add an empty meeting schedule.

Actual result:

When user tries to schedule a meeting that has nothing in its text fields, the app does not allows him to do so. There’s no dot on date that that was selected as shown in attached picture.

Status: Pass

Test case 22: Submission of meeting Data for multiple meetings

only enter and display from from database

Summary:

The test case will check behavior of the app when user tries to add a meeting schedule for multiple meetings

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Meeting schedule board” module.
4. Navigate to required date and select “Plus“ button to add meeting.
5. A pop up form appears, enter required data in text fields.
6. Press “Done“ button.

Test Data:

1st input:

Time: 2:53

Location: B-229

Topic: Orientation

Description: “Meetings of heads for discussing main event ideas“

2nd input:

Time: 2:53

Location: A-315

Topic: Qirat event

Description: “Meetings of heads for discussing main event ideas“

Expected Result:

The app should allow user to Add schedule of multiple meetings. It should be added in database.

Actual result:

When user tries to add schedule for multiple meeting , the app allows him to do so. App updates the user with popup message “Meeting added successfully in schedule“ when user selects “Done“ button.

Status: Pass

Test case 23: Scheduling meeting in past

Summary:

The test case will check behavior of the app when user tries to add a meeting schedule in past.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Meeting schedule board” module.
4. Navigate to date in past and select “Plus“ button to add meeting.
5. A pop up form appears, enter required data in text fields.
6. Press “Done“ button.

Test Data:

1st input:

Time: 2:53

Location: B-229

Topic: Orientation

Description: “Meetings of heads for discussing main event ideas“

2nd input:

Time: 2:53

Location: A-315

Topic: Qirat event

Description: “Meetings of heads for discussing main event ideas“

Expected Result:

The app should not allow user to Add schedule of multiple meetings. It should inform user by displaying a message “You can’t add meeting in past“

Actual result:

When user tries to add schedule for meeting in past, the app allows him to do so. App updates the user with popup message “Meeting added successfully in schedule“ when user selects “Done“ button and is added in database.

Status: Fail

Test case 24: Only president can schedule the meeting

Summary:

This test case is to check that only president can add a meeting schedule of a meeting.

Description:

1. Open Nimbus app.
2. Login using president’s id.
3. Go to “Meeting schedule board“ module and check if “plus(add)“ options present.
4. Open slider “Menu“ by clicking 3 bars at top left corner
5. Sign out from app by clicking “Sign out“
6. Now, login with member’s id
7. Go to “Meeting schedule board“ module and check if “plus(add)“options present.

Test Data:

1st entry:

- Email: kas@gmail.com
- Password: 12345678Aa$

2nd entry:

- Email: eman@gmail.com
- Password: 12345678Aa$

Expected Result:

When a president login then he should only see “plus(add)“ options in ”Meeting schedule board” module. If anyone other then president logs in then he should not see “plus(add)“ options in ”Meeting schedule board” module.

Actual result:

When a president login then he can see “plus(add)“ options in ”Meeting schedule board” module. While when a member logs in, he does not see “plus(add)“ options in ”Meeting schedule board” module.

Status: Pass

Test case 25: Scheduling multiple meetings on same date

Summary:

This test case is to check that multiple meetings can be seen on single date.

Description:

1. Open Nimbus app.
2. Go to “Meeting schedule board“ module and check if there are 2 or more dots on any date.
3. Select date and slide to see multiple schedule

Test Data:

- Email: eman@gmail.com
- Password: 12345678Aa$

Expected Result:

When user select a date having multiple dots on it, then it should display the total number of meeting schedule equal to total number of dots on that date.

Actual result:

When user select a date having multiple dots on it, then it displays the total number of meeting schedule equal to total number of dots on that date.

Status: Pass

Test case 26: Empty text fields Submission for New Member

Summary:

The test case will check behavior of the app when user tries to update a member without entering any data in text fields also without selecting date and time.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Member’s directory” module.
4. Select “Edit” button on a member data.
5. Leave all text fields empty and try to update the form.
6. Press “Update“ button.

Test Data:

No input (leave all fields empty).

Expected Result:

The app should not allow user to update empty member’s data.

Actual result:

When user tries to update member's data that has nothing in its text fields, the app allows him to do so. And data is updated in database

Status: Fail

Test case 27: Invalid email format update of memember

Summary:

This test case checks the behavior of Nimbus app when user tries to update data of member and enters email of invalid format.

Description:

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Select “Member's Directory“.
4. Select “Edit” button on the member’s data you want to update.
5. Enter an invalid email format.
6. Enter required fields

7. Press “Update“ button.

Test Data:

- Username: Sana
- Email: "sana@gmail"
- Password: "12345678Aa$"
- Role:
- Age:
- Post:

Expected Result:

When user tries to update member’s email that does not follow the criteria, the app should not allow it by displaying an error message “Invalid email format. Please follow format: example@domain.com“

Actual result:

When user tries to update member’s email that does not follow the criteria, the app allows it, and data is updated on database.

Status: Fail

Test case 28: Deletion of member

Summary:

The test case will check behavior of the app when user tries delete a member.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Member’s Directory” module.
4. Delete a specific event by pressing “Delete“ button.

Test Data:

No input (leave all fields empty).

Expected Result:

The app should allow user to delete member and displays remaining event on screen.

Actual result:

When user tries to delete an event, event is deleted from database, app updates user with the help of popup message “Member deleted successfully!“. Remaining events are then displayed on screen

Status: Pass

Test case 29: Update member information in database with correct data

Summary:

The test case will check behavior of the app when user tries to update an existing event.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Member’s directory” module.
4. Press “Edit” button on one of the member.
5. Rewrite the data that you want to update.
6. Press “Update“ button.

Test Data:

- Username: saim
- Email: "saim@gmail.com"
- Password: "12345678Aa$"
- Role: Non-society person
- Age: 20
- Post: Non-society person
- Phone number: 03123456789

Event Description: Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor elod wfyns wdym hseeh kucf ffo

Expected Result:

The app should allow user to Update data of an already existing member by updating it on database.

Actual result:

When user tries to update data of an already existed member, the app allows him to do so. When user press “Update” button event is added to database and a popup message informs user “Successfully updated Member!“.

Status: Pass

Test case 30: Cancel update of member's data

Test case 31: Successful creation of new member

Test case 32: Creating new member by submitting empty fields

Summary:

The test case will check behavior of the app when user tries to add a member without entering any data in text fields also without selecting date and time.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Member’s directory” module.
4. Select “Plus” button to add member.
5. Leave all text fields empty and try to update the form.
6. Press “Add“ button.

Test Data:

No input (leave all fields empty).

Expected Result:

The app should not allow user to update empty member’s data.

Actual result:

When user tries to update member's data that has nothing in its text fields, the app allows him to do so. And data is added in database. It updates user by popup message “Successfully added member!“.

Status: Fail

Test case 33: Invalid email format for creating new member

Summary:

This test case checks the behavior of Nimbus app when user tries to add a new member and enters email of invalid format.

Description:

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Select “Member's Directory“.
4. Select “Plus” button in the member’s directory you want to to add member.
5. Enter an invalid email format.
6. Enter required fields
7. Press “Update“ button.

Test Data:

- Username: Amna
- Email: "amna@gmail"
- Password: "12345678Aa$"
- Role: member
- Age: 20
- Post: member
- Phone number: 03123456789

Expected Result:

When user tries to add a new member and member’s email that user entered that does not follow the criteria, the app should not allow it by displaying an error message “Invalid email format. Please follow format: example@domain.com“

Actual result:

When user tries to create a new member and member’s email that user entered does not follow the criteria, the app allows it, and data is updated on database.

Status: Fail

Test case 34: Edit profile data

Summary:

The test case will check behavior of the app when user tries to edit his profile information.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Profile” module by clicking profile icon at top right corner.
4. Press “Edit” button on at top right corner of page.
5. Enter the data that you want to edit.
6. Press “Update“ button.

Test Data:

- Username: kashaf
- Email: "kas@gmail.com"
- Password: "12345678Aa$"
- Role: President
- Age: 25
- Post: President
- Phone number: 03123456789

Expected Result:

The app should allow user to Update his profile data by updating it on database.

Actual result:

When user tries to update his profile data , the app allows him to do so. When user press “Update” button member’s data is updated to database and a popup message informs user “Successfully updated Member’s data!“.

Status: Pass

Test case 35: Display correct user's data in profile

Summary:

The test case will check behavior of the app when user tries to open his profile information.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Profile” module by clicking profile icon at top right corner.

Test Data:

noting to enter

Expected Result:

The app should allow user to see his correct profile data by fetching it on database.

Actual result:

When user tries to see his profile data , the app allows him to do so. When user press “Profile” button correct data of user is fetched from database and shown in profile page.

Status: Pass

Test case 36: Change profile picture of user profile

Summary:

The test case will check behavior of the app when user tries to change his profile picture using camera.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Profile” module by clicking profile icon at top right corner.
4. Press “Edit” button on at top right corner of page.
5. Press “+“ button on profile image.
6. Select camera and take the image and mark it.
7. Press “Update“ button.

Test Data:

Take image from camera

Expected Result:

The app should allow user to Update his profile picture when he tries take a new picture using camera, by updating it on database.

Actual result:

When user tries to update his profile picture , the app allows him to do so. When user press “Update” button, profile picture is updated in database and a popup message informs user “Successfully updated Member’s data!“.

Status: Pass

Test case 37: Upload pictures of event in Event images in new folder

Summary:

The test case will check behavior of the app when user tries to upload event images from drive, by creating a new folder.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Event image” module .
4. Press “Add” button on at bottom right corner of page.
5. Select images from drive.
6. In popup menu enter new folder name.
7. Press “Submit“ button.

Test Data:

Select images from drive

Expected Result:

The app should allow user to Upload images from his drive, app should create a new folder and add images in it. Also it should upload images on database

Actual result:

The app allows user to Upload images from his drive, app creates a new folder and upload images in it. Also upload images on database

Status: Pass

Test case 38: Download zip folder of a folder of event images

Summary:

The test case will check behavior of the app when user tries to download event image folder

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Event images” module .
4. Press “Download” icon on the right side of any folder

Test Data:

Select folder to download

Expected Result:

The app should allow user to download images from the app, app should download a zip folder of the event image folder that user selected.

Actual result:

The app allows user to download images from the app. It downloads a zip folder of the event image folder that user selected.

Status: Pass

Test case 39: Upload images of event in already existed folder

Summary:

The test case will check behavior of the app when user tries to upload event images from drive in already existed folder on database.

Description:

1. Open Nimbus app
2. Sign up in app.
3. Go to “Event image” module .
4. Press “Add” button on at bottom right corner of page.
5. Select images from drive.

6. In popup menu enter folder name of existing folder in which you want to add new images.
7. Press “Submit“ button.

Test Data:

Select images from drive

Expected Result:

The app should allow user to Upload images from his drive, app should add images in already existed folder of which user enters the name . Also it should upload images on database

Actual result:

The app allows user to Upload images from his drive. App adds images in already existed folder of which user enters the name . Also it uploads images on database. It updates user by showing popup message “Images uploaded successfully!!“.

Status: Pass

Test case 40: Only president and Mentor can upload images of events

Summary:

This test case will confirm that only president and mentor can upload pictures of an event.

Description:

1. Open Nimbus app
2. Sign up in app as president.
3. Go to “Event image” module .
4. Check if there is “Add“ button at the right bottom.
5. Sign out from the app by clicking “Sign out“ button in sliding menu bar.
6. Sign up in app as member.
7. Go to “Event image” module .
8. Check if there is “Add“ button at the right bottom.

Test Data:

1st input:

- Email: kas@gmail.com
- Password: 12345678Aa$

2nd input:

- Email: eman@gmail.com
- Password: 12345678Aa$

Expected Result:

The app should only allow president and mentor to upload image by giving “Upload“ option to president and mentor only.

Actual result:

The app only allow president and mentor to upload image by giving “Upload“ option to president and mentor only. We can see it in attached images.

Status: Pass

Test case 41: Approval of event request by president

Summary:

The test case will check the Approval status of an event request when users approve it. It checks when user approve the request does it’s approval status changes or not.

Description:

1. Open Nimbus app
2. Login as president
3. Go to “Approve event request“ module.
4. Press “Approve“ button to approve event requests.

Test Data:

Expected Result:

The app should update Approval status of the event in database when user approves the event.

Actual result:

When user approves the event, then app updates the Approval status of event true in database as shown in attached picture. It updates user by popup message saying “Event approved successfully!!“.

Status: Pass

Test case 42: Approve event participation request

Summary:

The test case will check the Approval status of an event participation request of students, when users approve it. It checks when user approve the request does it's approval status changes or not in database.

Description:

1. Open Nimbus app
2. Login as president
3. Go to "Approve event participation request" module.
4. Press "Approve" button to approve event requests.

Test Data:

Expected Result:

The app should update Approval status of the event participation request of other users in database when user approves the event.

Actual result:

When user approves the event participation request, then app updates the Approval status of event true in database as shown in attached picture. It updates user by popup message saying "Application approved successfully!!".

Status: Pass

Test case 43: Event proposal request

Summary:

The test case will check the behavior of app when user wants to propose an event to occur.

Description:

1. Open Nimbus app
2. Login in the app
3. Go to "Add event request" module.
4. Enter all the data required.

5. Press “Send“ button.

Test Data:

Budget: 12000

Date: 2024-09-20

Description: hre is the descriptipn

Event Name: Event 1

Venue: A-315

Time: 3:00 PM

Organizer: FSES

Expected Result:

The app should allow user to send a request of event proposal. It should let user know that request has sent when user press “Send“ by popup message “Event request sent successfully“

Actual result:

The app allows user to send a request of event proposal. It lets user know that request has sent when user press “Send“ by popup message “Event request sent successfully“

Status: Pass

Test case 44: Empty form submission of event participation request

Summary:

The test case will check the behavior of app when user wants to apply for the participation in the event without filling any text fields of form.

Description:

1. Open Nimbus app
2. Login in the app
3. Go to “Event participation request“ module.
4. Press “Send“ button.

Test Data:

Let text fields remain empty.

Expected Result:

The app should not allow user to send an empty request for event participation. It should let user know that request hasn't been sent when user press "Send" by popup message "Please enter all required information"

Actual result:

The app allows user to send an empty request for event participation. It lets user know that request has sent when user press "Send" by popup message "Participation request sent successfully!!"

Status: Fail

Test case 45: Empty form submission of event proposal request

Summary:

The test case will check the behavior of app when user wants to propose an event to occur.

Description:

1. Open Nimbus app
2. Login in the app
3. Go to "Add event request" module.
4. Press "Send" button.

Test Data:

Let text feels remain empty.

Expected Result:

The app should not allow user to send an empty request of event proposal. It should let user know that request hasn't been sent when user press "Send" by popup message "Please enter all required information"

Actual result:

The app allows user to send an empty request of event proposal. It lets user know that request has sent when user press “Send“ by popup message “Event request sent successfully“

Status: Fail

Test case 46: Event participation request

Summary:

The test case will check the behavior of app when user wants to apply for the participation in the event.

Description:

1. Open Nimbus app
2. Login in the app
3. Go to “Event participation request“ module.
4. Enter all the data required.
5. Press “Send“ button.

Test Data:

Batch: 22

Email: saim@gmail.com

First name: Saim

Last name: Nevaan

Phone number: 03123456789

Expected Result:

The app should allow user to send a request for event participation. It should let user know that request has sent when user press “Send“ by popup message “Participation request sent successfully“

Actual result:

The app allows user to send a request for event participation. It lets user know that request has sent when user press “Send“ by popup message “Participation request sent successfully!!”

Status: Pass

Test case 47: Event proposal request in past

Summary:

The test case will check the behavior of app when user wants to propose an event to occur in past.

Description:

1. Open Nimbus app
2. Login in the app
3. Go to “Add event request“ module.
4. Enter all the data required.
5. Set date from past.
6. Press “Send“ button.

Test Data:

Budget: 10000

Date: 2024-09-03

Description: description is here

Event Name: Past Event

Venue: B-229

Time: 4:00 PM

Organizer: FSES

Expected Result:

The app should not allow user to send a request of event proposal in past. It should let user know that request hasn't been sent when user press “Send“ by popup message “Event can’t be created in past“

Actual result:

The app allows user to send a request of event proposal in past. It lets user know that request has sent when user press “Send“ by popup message “Event request sent successfully“

Status: Fail

3. Summary of critical bugs found, their impact on the application, and proposed solutions

Bug 1: Email sent for reset password on which account does not exists

Summary:

The app accidently sends reset password of an email on which account does not exists.

Description:

The user tries to reset password of an email on which account does not exists. When user presses "Send reset link", the app mistakenly sends password reset email even though no account exists on that email.

- **Steps to reproduce:**

1. Open Nimbus app
2. Press "Log in" button to navigate to login page
3. Press "forget password" button
4. Enter email in text field and press "Send reset link"

(A reset link will be sent in Email)

Affected component:

- Authentication (Reset password feature)

Severity: High

Bug 2: Email of invalid format Allows Sign-Up Without Error Notification

Summary:

The app let user to sign in with an invalid email format and fails to display the error message to user. Hence it results in registration of user in database

Description:

When user tries to signup using an email that does not follow requirements, app allows it and user is added in database. App also does not notify any issue to user by displaying any error message.

- **Steps to Reproduce:**

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Enter valid username , email and password.
4. Select the role from drop down menu.
5. Press “Sign up“ button.

Affected component: Sign-Up (Email Validation)

Severity: High

Bug 3: Events added without data in the required fields

Summary:

The app lets user to add an event that has nothing in its text fields.

Description:

When user tries to add an event that has nothing in its text fields, the app allows him to do so. When user press “Add” button event is added to database and a popup message informs user “Event is added!“.

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Leave all text fields empty and try to submit the form.
5. Press “Add“ button.

Affected component:

- Authentication (Reset password feature)

Severity: High

Bug 4: Event is added in past by user

Summary:

The app allows users to add events in past. It should not be permitted and error message must be displayed.

Description:

When user tries to add an event in past by choosing a previous date from present, app allows him to do so. Event is then added in database.

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Select a date from past and tries to submit it.
5. Press “Add“ button.

Affected component:

- Event creation module (Date validation)

Severity: High

Priority: Medium

Bug 5: Event Description Field Word Limit Issue

Summary:

A text field “Event description” in “Add event“ module allow only 73 words submission but its limit is 500.

Description:

When user tries to enter more than 500 words in “Event description“ in Add event module, user can only enter 73 words as shown in the picture. Text field does not allow to enter further words

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Enter 502 words in “Event Description“ text field.
5. Enter valid data in all text fields that is required.

6. Press “Add“ button.

Affected component:

- Event Creation Module ("Event Description" Text Field)

Severity: Medium

Priority: Medium

Bug 6: Non-Numeric Data Allowed in "Budget" Field

Summary:

The text field “Budget“ allows user to enter non-numeric data in it.

Description:

When user tries to enter non-numeric data in “Budget“ in Add event module, App allows user to do so and event is then saved in data base as shown in pictures.

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Add non-numeric data in “Budget“ text field.
5. Enter valid data in all text fields that is required.
6. Press “Add“ button.

Affected component:

- Event Creation Module ("Budget" Text Field)

Severity: High

Bug 7: Numeric Data Allowed in "Event Name" Field

Summary:

The text field "Event Name" in "Add Event" module allows user to enter numeric data in it, which should be prohibited.

Description:

When user tries to enter numeric data in “Event name“ in Add event module, App allows user to do so and event is then saved in data base as shown in pictures.

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Add event” module.
4. Add numeric data in “Event name“ text field.
5. Enter valid data in all text fields that is required.
6. Press “Add“ button.

Affected component:

- Event Creation Module ("Event Name" Text Field)

Severity: Medium

Bug 8: Event Update with Empty Fields Allowed

Summary:

The app let user to edit text field and update it even though some required text fields are empty.

Description:

When user tries to update an event that has empty text fields, the app allows him to do so. When user press “Update” button event is added to database and a popup message informs user “Successfully updated event!“.

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “View event” module.
4. Press “Edit” button on one of the event.
5. Enter data in few fields and leave few empty.
6. Press “Update“ button.

Affected component:

- Event Update Module

Severity: High

Bug 9: Adding Meeting Schedule in the Past

Summary:

The app allows user to set meetings schedule in past date, which is not technically correct.

Description:

When user tries to add schedule for meeting in past, the app allows him to do so. App updates the user with popup message “Meeting added successfully in schedule“ when user selects “Done“ button and is added in database.

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Meeting schedule board” module.
4. Navigate to date in past and select “Plus“ button to add meeting.
5. A pop up form appears, enter required data in text fields.
6. Press “Done“ button.

Affected component:

- Meeting Schedule Board Module

Severity: High

Bug 10: Updating Member Data with Empty Fields

Summary:

The app allows users to update member data with empty fields, which should be restricted.

The app allows user to update data without entering any data in required text fields, it should be restricted.

Description:

When user tries to update member's data that has nothing in its text fields, the app allows him to do so. And data is updated in database

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Member’s directory” module.
4. Select “Edit” button on a member data.
5. Leave all text fields empty and try to update the form.
6. Press “Update“ button.

Affected component:

- Member’s Directory Module

Severity: High

Bug 11: Invalid Email Format Allowed During Member Data Update

Summary:

The app allows users to update member data using an email that does not have the required format, which should be restricted.

Description:

When user tries to update member’s email that does not follow the criteria, the app allows it, and data is updated on database.

- **Steps to reproduce:**

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Select “Member's Directory“.
4. Select “Edit” button on the member’s data you want to update.
5. Enter an invalid email format.
6. Enter required fields
7. Press “Update“ button.

Affected component:

- Member’s Directory Module (Email Validation)

Severity: High

Bug 12: Empty Member Data Allowed During Add Member Operation

Summary:

The app allows users to add a member even if required fields are empty, which should be restricted.

Description:

When user tries to update member's data that has nothing in its text fields, the app allows him to do so. And data is added in database. It updates user by popup message “Successfully added member!“.

- **Steps to reproduce:**

1. Open Nimbus app
2. Sign up in app.
3. Go to “Member’s directory” module.
4. Select “Plus” button to add member.
5. Leave all text fields empty and try to update the form.
6. Press “Add“ button.

Affected component:

- Member’s Directory Module (Form Validation for Adding Member)

Severity: High

Bug 13: Invalid Email Format Accepted During Add Member Operation

Summary:

The app allows users to add a new member with an incorrectly formatted email, which should be restricted.

Description:

When user tries to create a new member and member’s email that user entered does not follow the criteria, the app allows it, and data is updated on database.

- **Steps to reproduce:**

1. Open Nimbus app.
2. Press “Sign up” button to go to sign up page.
3. Select “Member's Directory“.
4. Select “Plus” button in the member’s directory you want to to add member.
5. Enter an invalid email format.
6. Enter required fields
7. Press “Update“ button.

Affected component:

- Member’s Directory Module (Email Validation for Adding Member)

Severity: High

Bug 14: Empty Event Participation Request Form Submission Allowed

Summary:

The app allows users to send an empty participation request for an event, which should be restricted.

Description:

The app allows user to send an empty request for event participation. It lets user know that request has sent when user press “Send“ by popup message “Participation request sent successfully!!”

- **Steps to reproduce:**

1. Open Nimbus app
2. Login in the app
3. Go to “Event participation request“ module.
4. Press “Send“ button.

Affected component:

- Event Participation Request Form (Validation Check for Empty Fields)

Severity: High

Bug 15: Empty Event Proposal Request Form Submission Allowed

Summary:

The app permits users to send an empty event proposal request, which should be prevented.

Description:

The app allows user to send an empty request of event proposal. It lets user know that request has sent when user press “Send“ by popup message “Event request sent successfully“

- **Steps to reproduce:**

1. Open Nimbus app
2. Login in the app
3. Go to “Add event request“ module.
4. Press “Send“ button.

Affected component:

- Event Proposal Request Form (Validation Check for Empty Fields)

Severity: High

Bug 16: Event Proposal Request Allows Past Date Submission

Summary:

The app permits users to propose events in the past, which should not be allowed.

Description:

The app allows user to send a request of event proposal in past. It lets user know that request has sent when user press “Send“ by popup message “Event request sent successfully“

- **Steps to reproduce:**

1. Open Nimbus app
2. Login in the app
3. Go to “Add event request“ module.
4. Enter all the data required.
5. Set date from past.
6. Press “Send“ button.

Affected component:

- Event Proposal Request Form (Date Validation)

Severity: High

5. What I learned from this assignment?

I learned the use of Jira tool , how we can create test cases, how we can create bugs when test cases are failed. I also learned how to link bugs with test cases when test cases are failed. We should be patient in doing testing. I also learned how to do black box testing of my flutter project.